

# **Project 1**

By

Vimlesh Sonawane

## **Introduction to Analytics**

**ALY6000**

Professor Wada Roy



MS in Project Management

Northeastern University

(College of Professional Studies)

April 2025

## 1.1. Introduction

This report summarizes the work completed for Project 1, which was designed to strengthen foundational skills in R programming. The tasks included performing simple arithmetic, creating and modifying vectors, applying logical expressions, and generating basic statistical summaries. The project also involved analyzing a real-world dataset (ds\_salaries.csv) to explore insights related to job roles, experience levels, and salary trends within the data science industry.

To support the analysis, visualizations were created using the ggplot2 package in R. These included bar charts and histograms that illustrated patterns in salary distribution, differences among various job roles, and how experience level impacts compensation. These visual tools made it easier to interpret the data and uncover meaningful trends in the field.

## 2.1. Calculations

### Problem 1 :

Write lines of code to compute all of the following. Include the answers in your written report.

#### Script :

```
# Problem 1
# Multiplication
result1 <- (123 * 453)
print(result1)

# Exponentiation and Multiplication
result2 <- (5^2 * 40)
print(result2)

# Logical AND
result3 <- (TRUE & FALSE)
print(result3)

# Logical OR
result4 <- (TRUE | FALSE)
print(result4)

# Modulus
result5 <- (75 %% 10)
print(result5)

# Division
result6 <- (75 / 10)
print(result6)
```

#### Output:

```

> # Problem 1
> # Multiplication
> result1 <- (123 * 453)
> print(result1)
[1] 55719
>
> # Exponentiation and Multiplication
> result2 <- (5^2 * 40)
> print(result2)
[1] 1000
>
> # Logical AND
> result3 <- (TRUE & FALSE)
> print(result3)
[1] FALSE
>
> # Logical OR
> result4 <- (TRUE | FALSE)
> print(result4)
[1] TRUE
>
> # Modulus
> result5 <- (75 %% 10)
> print(result5)
[1] 5
>
> # Division
> result6 <- (75 / 10)
> print(result6)
[1] 7.5
>

```

## Problem 2 :

Create a vector using the c function with the values 17, 12, -33, 5 and assign it to a variable called first\_vector.

### Script:

```

30
31 # Problem 2
32 # Create a vector using the c function with the values 17, 12, -33, 5 and assign it to a variable called first_vector
33 first_vector <- c(17, 12, -33, 5)
34 # print the vector
35 print(first_vector)
36

```

### Output:

```

> # Problem 2
> # Create a vector using the c function with the values 17, 12, -33, 5 and assign it to a variable called first_vector.
> first_vector <- c(17, 12, -33, 5)
> # print the vector
> print(first_vector)
[1] 17 12 -33 5
>

```

## Problem 3 :

Create a vector using the c function with the values 5, 10, 15, 20, 25, 30, 35 and assign it to a variable called counting\_by\_fives.

### Script:

```

# Problem 3
counting_by_fives <- c(5, 10, 15, 20, 25, 30, 35)
print(counting_by_fives)

```

### Output:

```
>
> # Problem 3
> counting_by_fives <- c(5, 10, 15, 20, 25, 30, 35)
> print(counting_by_fives)
[1] 5 10 15 20 25 30 35
```

### Problem 4:

Create a vector using the range operator (the colon), that contains the numbers from 20 down to 1. Store the result in a variable called `second_vector`.

### Script:

```
# Problem 4
second_vector <- 20:1
print(second_vector)
```

### Output:

```
> # Problem 4
> second_vector <- 20:1
> print(second_vector)
[1] 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

### Problem 5:

Create a vector using the range operator that contains the number from 5 to 15. Store the result in a variable called `counting_vector`

### Script:

```
# Problem 5
counting_vector <- 5:15
print(counting_vector)
```

### Output:

```
> # Problem 5
> counting_vector <- 5:15
> print(counting_vector)
[1] 5 6 7 8 9 10 11 12 13 14 15
```

### Problem 6:

Create a vector with the values (96, 100, 85, 92, 81, 72). Store the result in a variable called `grades`

### Script:

```
# Problem 6
grades <- c(96, 100, 85, 92, 81, 72)
print(grades)

# Problem 7
```

### Output:

```
> # Problem 6
> grades <- c(96, 100, 85, 92, 81, 72)
> print(grades)
[1] 96 100 85 92 81 72
~
```

### Problem 7:

Add the number 3 to the vector grades. Store the result in a variable called bonus\_points\_added.

### Script:

```
# Problem 7
bonus_points_added <- grades + 3
print(bonus_points_added)
```

### Output:

```
> # Problem 7
> bonus_points_added <- grades + 3
> print(bonus_points_added)
[1] 99 103 88 95 84 75
~
```

### Problem 8:

Create a vector one\_to\_one\_hundred with values 1 through 100.

### Script:

```
# Problem 8
one_to_one_hundred <- 1:100
print(one_to_one_hundred)
```

### Output:

```
> # Problem 8
> one_to_one_hundred <- 1:100
> print(one_to_one_hundred)
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
[30] 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58
[59] 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87
[88] 88 89 90 91 92 93 94 95 96 97 98 99 100
>
```

### Problem 9:

Perform the following operations on second\_vector:

- Add 20
- Multiply by 20
- Check for values  $\geq 20$
- Check for values  $\neq 20$

### Script:

```
# Problem 9
# Add 20 to each element of second_vector
print(second_vector + 20)

# Multiply each element of second_vector by 20
print(second_vector * 20)

# Return TRUE if element is greater than or equal to 20
print(second_vector >= 20)

# Return TRUE if element is not equal to 20
print(second_vector != 20)
```

### Output:

```
> # Problem 9
> # Add 20 to each element of second_vector
> print(second_vector + 20)
[1] 40 39 38 37 36 35 34 33 32 31 30 29 28 27 26 25 24 23 22 21
>
> # Multiply each element of second_vector by 20
> print(second_vector * 20)
[1] 400 380 360 340 320 300 280 260 240 220 200 180 160 140 120 100 80 60 40 20
>
> # Return TRUE if element is greater than or equal to 20
> print(second_vector >= 20)
[1] TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[20] FALSE
>
> # Return TRUE if element is not equal to 20
> print(second_vector != 20)
[1] FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[20] TRUE
```

### Problem 10:

Calculate the sum of one\_to\_one\_hundred and store in total.

### Script:

```
# Problem 10
total <- sum(one_to_one_hundred)
print(total)
```

### Output:

```
> # Problem 10
> total <- sum(one_to_one_hundred)
> print(total)
[1] 5050
>
```

### Problem 11:

Find the mean of one\_to\_one\_hundred.

### Script:

```
# Problem 11
average_value <- mean(one_to_one_hundred)
print(average_value)
```

**Output:**

```
> # Problem 11
> average_value <- mean(one_to_one_hundred)
> print(average_value)
[1] 50.5
```

**Problem 12:**

Find the median of one\_to\_one\_hundred.

**Script:**

```
# Problem 12
median_value <- median(one_to_one_hundred)
print(median_value)
```

**Output:**

```
> # Problem 12
> median_value <- median(one_to_one_hundred)
> print(median_value)
[1] 50.5
```

**Problem 13:**

Find the maximum value in one\_to\_one\_hundred.

**Script:**

```
# Problem 13
max_value <- max(one_to_one_hundred)
print(max_value)
```

**Output:**

```
>
> # Problem 13
> max_value <- max(one_to_one_hundred)
> print(max_value)
[1] 100
>
```

**Problem 14:**

Find the minimum value in one\_to\_one\_hundred.

**Script:**

```
# Problem 14
min_value <- min(one_to_one_hundred)
print(min_value)
```

**Output:**

```
> # Problem 14
> min_value <- min(one_to_one_hundred)
> print(min_value)
[1] 1
>
```

### Problem 15:

Extract the first value from second\_vector.

#### Script:

```
# Problem 15
first_value <- second_vector[1]
print(first_value)
```

#### Output:

```
> # Problem 15
> first_value <- second_vector[1]
> print(first_value)
[1] 20
```

### Problem 16:

Extract the first three values from second\_vector.

#### Script:

```
# Problem 16
first_three_values <- second_vector[1:3]
print(first_three_values)
```

#### Output:

```
> # Problem 16
> first_three_values <- second_vector[1:3]
> print(first_three_values)
[1] 20 19 18
```

### Problem 17:

Use c() to extract the 1st, 5th, 10th, and 11th elements of second\_vector.

#### Script:

```
# Problem 17
vector_from_brackets <- second_vector[c(1, 5, 10, 11)]
print(vector_from_brackets)
```

#### Output:

```
> # Problem 17
> vector_from_brackets <- second_vector[c(1, 5, 10, 11)]
> print(vector_from_brackets)
[1] 20 16 11 10
```

### Problem 18:

Use a Boolean vector to extract only the 2nd and 4th values from first\_vector.

#### Script:

```
# Problem 18
vector_from_boolean_brackets <- first_vector[c(FALSE, TRUE, FALSE, TRUE)]
# Only elements with TRUE in the logical vector are extracted
print(vector_from_boolean_brackets)
```

### Output:

```
> # Problem 18
> vector_from_boolean_brackets <- first_vector[c(FALSE, TRUE, FALSE, TRUE)]
> # Only elements with TRUE in the logical vector are extracted
> print(vector_from_boolean_brackets)
[1] 12 5
```

### Problem 19:

Check which values in second\_vector are greater than or equal to 10.

### Script:

```
# Problem 19
# This returns a logical vector indicating whether each element of second_vector is greater than or equal to 10
print(second_vector >= 10)
```

### Output:

```
> # Problem 19
> # This returns a logical vector indicating whether each element of second_vector is greater than or equal to 10
> print(second_vector >= 10)
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[20] FALSE
```

### Problem 20:

From one\_to\_one\_hundred, display only values that are greater than or equal to 20.

### Script:

```
# Problem 20
# This returns only the values from one_to_one_hundred that are greater than or equal to 20
print(one_to_one_hundred[one_to_one_hundred >= 20])
```

### Output:

```
> # Problem 20
> # This returns only the values from one_to_one_hundred that are greater than or equal to 20
> print(one_to_one_hundred[one_to_one_hundred >= 20])
[1] 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
[30] 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77
[59] 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

### Problem 21:

Remove all grades less than or equal to 85 from grades.

### Script:

```
# Problem 21
lowest_grades_removed <- grades[grades > 85]
print(lowest_grades_removed)
```

### Output:

```
> # Problem 21
> lowest_grades_removed <- grades[grades > 85]
> print(lowest_grades_removed)
[1] 96 100 92
```

## Problem 22:

Remove the 3rd and 4th values from grades.

### Script:

```
# Problem 22
middle_grades_removed <- grades[-c(3, 4)]
print(middle_grades_removed)
```

### Output:

```
> # Problem 22
> middle_grades_removed <- grades[-c(3, 4)]
> print(middle_grades_removed)
[1] 96 100 81 72
```

## Problem 23:

Remove the 5th and 10th values from second\_vector and store it in fifth\_vector.

### Script:

```
# Problem 23
fifth_vector <- second_vector[-c(5, 10)]
print(fifth_vector)
```

### Output:

```
> # Problem 23
> fifth_vector <- second_vector[-c(5, 10)]
> print(fifth_vector)
[1] 20 19 18 17 15 14 13 12 10 9 8 7 6 5 4 3 2 1
```

## Problem 24:

Use runif() to create a vector random\_vector with 10 random numbers between 0 and 1000. Set seed to 5.

### Script:

```
# Problem 24
set.seed(5)
random_vector <- runif(n = 10, min = 0, max = 1000)
print(random_vector)
```

### Output:

```
> # Problem 24
> set.seed(5)
> random_vector <- runif(n = 10, min = 0, max = 1000)
> print(random_vector)
[1] 200.2145 685.2186 916.8758 284.3995 104.6501 701.0575 527.9600 807.9352 956.5001 110.4530
```

### Problem 25:

Find the sum of random\_vector and store it in sum\_vector.

#### Script:

```
# Problem 25
sum_vector <- sum(random_vector)
print(sum_vector)
```

#### Output:

```
> # Problem 25
> sum_vector <- sum(random_vector)
> print(sum_vector)
[1] 5295.264
```

### Problem 26:

Compute the cumulative sum of random\_vector and store in cumsum\_vector.

#### Script:

```
# Problem 26
cumsum_vector <- cumsum(random_vector)
print(cumsum_vector)
```

#### Output:

```
> # Problem 26
> cumsum_vector <- cumsum(random_vector)
> print(cumsum_vector)
[1] 200.2145 885.4330 1802.3088 2086.7083 2191.3584 2892.4159 3420.3759 4228.3111 5184.8112 5295.2642
```

### Problem 27:

Calculate the mean of random\_vector.

#### Script:

```
# Problem 27
mean_vector <- mean(random_vector)
print(mean_vector)
```

#### Output:

```
> # Problem 27
> mean_vector <- mean(random_vector)
> print(mean_vector)
[1] 529.5264
```

### Problem 28:

Calculate the standard deviation of random\_vector.

#### Script:

```
# Problem 28
sd_vector <- sd(random_vector)
print(sd_vector)
```

**Output:**

```
> # Problem 28
> sd_vector <- sd(random_vector)
> print(sd_vector)
[1] 331.3606
```

**Problem 29:**

Round all values in random\_vector.

**Script:**

```
# Problem 29
round_vector <- round(random_vector)
print(round_vector)
```

**Output:**

```
> # Problem 29
> round_vector <- round(random_vector)
> print(round_vector)
[1] 200 685 917 284 105 701 528 808 957 110
```

**Problem 30:**

Sort random\_vector in ascending order and store it in sort\_vector.

**Script:**

```
# Problem 30
sort_vector <- sort(random_vector)
print(sort_vector)
```

**Output:**

```
> # Problem 30
> sort_vector <- sort(random_vector)
> print(sort_vector)
[1] 104.6501 110.4530 200.2145 284.3995 527.9600 685.2186 701.0575 807.9352 916.8758 956.5001
```

**Problem 31:**

Load the dataset ds\_salaries.csv into first\_dataframe.

**Script:**

```
# Problem 31
# Download the datafile ds\_salaries.csv from Canvas. Save it in the same folder as your R file.
```

**Output:**

```
> # Problem 31
> # Download the datafile ds_salaries.csv from Canvas. Save it in the same folder as your R file.
```

## Problem 32:

Use summary() to explore first\_dataframe.

### Script:

```
# Problem 32
# Load the dataset ds_salaries.csv
first_dataframe <- read.csv("ds_salaries.csv")
summary(first_dataframe)
```

### Output:

```
> # Problem 32
> # Load the dataset ds_salaries.csv
> first_dataframe <- read.csv("ds_salaries.csv")
> summary(first_dataframe)
      X      work_year  experience_level  employment_type  job_title      salary
Min.   : 0.0   Min.   :2020   Length:607   Length:607   Length:607   Min.    : 4000
1st Qu.:151.5 1st Qu.:2021   Class :character   Class :character   Class :character   1st Qu.: 70000
Median :303.0 Median :2022   Mode  :character   Mode  :character   Mode  :character   Median : 115000
Mean   :303.0 Mean   :2021                                           Mean   : 324000
3rd Qu.:454.5 3rd Qu.:2022                                           3rd Qu.: 165000
Max.   :606.0 Max.   :2022                                           Max.   :30400000

salary_currency  salary_in_usd  employee_residence  remote_ratio  company_location  company_size
Length:607      Min.    : 2859   Length:607        Min.    : 0.00   Length:607        Length:607
Class :character 1st Qu.: 62726   Class :character   1st Qu.: 50.00   Class :character   Class :character
Mode  :character Median :101570   Mode  :character   Median :100.00   Mode  :character   Mode  :character
                  Mean   :112298                      Mean   : 70.92
                  3rd Qu.:150000                      3rd Qu.:100.00
                  Max.    :600000                      Max.    :100.00
```

## Problem 33:

Create a histogram of salary\_in\_usd and a barplot of experience\_level.

### Script:

```
# Problem 33
# Use the summary function with first_dataframe to produce summary statistics
summary_stats <- summary(first_dataframe)
print(summary_stats)
```

### Output:

```
> # Problem 33
> # Use the summary function with first_dataframe to produce summary statistics
> summary_stats <- summary(first_dataframe)
> print(summary_stats)
      X      work_year  experience_level  employment_type  job_title      salary
Min.   : 0.0   Min.   :2020   Length:607   Length:607   Length:607   Min.    : 4000
1st Qu.:151.5 1st Qu.:2021   Class :character   Class :character   Class :character   1st Qu.: 70000
Median :303.0 Median :2022   Mode  :character   Mode  :character   Mode  :character   Median : 115000
Mean   :303.0 Mean   :2021                                           Mean   : 324000
3rd Qu.:454.5 3rd Qu.:2022                                           3rd Qu.: 165000
Max.   :606.0 Max.   :2022                                           Max.   :30400000

salary_currency  salary_in_usd  employee_residence  remote_ratio  company_location  company_size
Length:607      Min.    : 2859   Length:607        Min.    : 0.00   Length:607        Length:607
Class :character 1st Qu.: 62726   Class :character   1st Qu.: 50.00   Class :character   Class :character
Mode  :character Median :101570   Mode  :character   Median :100.00   Mode  :character   Mode  :character
                  Mean   :112298                      Mean   : 70.92
                  3rd Qu.:150000                      3rd Qu.:100.00
                  Max.    :600000                      Max.    :100.00
```

### 3. Visualization

#### Script:

```
# Visualization
# Histogram : Salary Distribution
hist(first_dataframe$salary_in_usd, main="Salary Distribution", xlab="Salary (USD)", col="skyblue")

# Barplot : Experience Level Count
barplot(table(first_dataframe$experience_level), main="Experience Level Count", col="orange")
```

#### Output:

